

AREA SCIENCE PARK
RESEARCH AND TECHNOLOGY INSTITUTE
LABORATORY OF DATA ENGINEERING

Web Application for DPCfam and DPCstruct Data Exploration

Report 6 (15 Feb - 23 Feb 2026) : Web Development
(DPCStruct Frontend)

Emmanuel Nyandu Kagarabi
Master in Data Management and Curation (MDMC - SISSA)

Supervisors :
Valerio Pionponi
Elaheh Saadat

February 23, 2026

1 Introduction

This report is the sixth in a sequence documenting the weekly assigned tasks of our internship. Focusing on the DPCStruct dataset ([Barone et al., 2025](#)), we have reviewed the frontend content previously suggested in Report 2.

2 Development

2.1 Generalities on DPCStruct

We refer to **DPCStruct** as a dataset obtained by running an unsupervised algorithm (**DPCStruct**) designed to classify protein domains using AlphaFold2 predictions, developed by [Barone et al. \(2025\)](#) on a dataset comprising roughly 15 million proteins, representative of the **Foldseek** Clusters dataset. This dataset is a fully clustered version of the **AlphaFold** dataset, which contains roughly 214 million predicted protein structures. The final output consisted of 1, 642,061 domains distributed over 1,252,179 proteins, organised into 28,246 metaclusters based on their structural similarity.

Additionally, [Barone et al. \(2025\)](#) compared their results with some existing tools that explore proteins at a domain level. Namely,

- **PFAM** : Which identifies protein domains based on conserved amino acid sequences. Proteins are then classified into families(PF), and some families that are evolutionarily related are then grouped into Clans (CL). 70% of DPCStruct metaclusters showed perfect consistency with Pfam clans.
- **CATH** (Class,Architecture,Topology,Homology) : is a hierarchical database (Class-Architecture-Topology-Homologous Superfamily) used to classify protein domain structures from the Protein Data Bank(PDB), focusing on geometric/structural similarities. The DPCStruct algorithm recovered 83% of CATH folds.
- **SCOP** (Structural Classification of Proteins) : like CATH, it is also a hierarchical database (Class-Fold-Superfamily-Family) used to classify protein domain structures from the Protein Data Bank(PDB), focusing on evolutionary relationships(homology). The DPCStruct algorithm recovered 91% of SCOP folds.

24% of DPCStruct metaclusters showed no similarity to SCOP, CATH, or Pfam.

After downloading DPCStruct data from Zenodo, we previously presented in Report 1 a list of available files:

1. `dpcstruct_classification.tsv`:
2. `mcs_reps.fasta`
3. `mcs_reps_pdbs.zip`,
4. `mcs_properties.tsv`,
5. `annotated_cath_qc0.5_t0.5_10.5.tsv`
6. `annotated_cath_qc0.8_t0.5_10.5.tsv`
7. `annotated_scop_qc0.5_t0.5_10.5.tsv`

8. annotated_scop_qc0.8_t0.5_10.5.tsv
9. dpcstruct_consistency.tsv
10. pfam_consistency.tsv

2.2 Frontend Organization

The only interesting question is : **How can we properly organise the 10 files enumerated above and include optimal information in our web app?**

In answering our main question, we can extract meaningful information from those files through some tables as done in DPCFam :

Table 1: **DPCStruct Metaclusters Properties** - Combining files 4 and 9.

mc_id	size	len_aa	len_std	len_ratio	plddt	tmscore	pident	pfam_score	pfam_labels
MC0	43	112.02	12.36	0.11	78.53	0.54	23.74	—	—
—	-	-	-	-	-	-	-	-	-
MC5	46	159.04	38.84	0.24	81.34	0.56	17.34	1.00	CL0263
—	-	-	-	-	-	-	-	-	-
MC64574	11	90.36	9.42	0.10	86.57	0.87	39.65	—	—

We have 28,246 samples from **MC0** to **MC64574**, but **14**, **423** are represented in PFam. We use :

- **mc_id**: unique identifier per metacluster : MC0,...
- **size**: Number of domains per metacluster.
- **len_aa**: Average length of domains (number of amino acids).
- **len_std**: Standard deviation of domain lengths.
- **len_ratio**: Ratio of **len_std** to **len_aa**.
- **plddt**: Average Predicted Local Distance Difference Test (pLDDT) as reported by AlphaFold2. This value represents how confident AlphaFold is in estimating the 3D structure of a protein; the higher, the better.
- **tmscore**: Pairwise alignment Template Modeling (TM)-score between domains, averaged over all pairs, using the maximum TM-score normalized by query or target.
Essentially, the TM score (0 to 1) represents a global topological similarity of proteins measuring how similar their 3D folds are. In our particular case, it indicates how similar the structures within this Metacluster are (High = very tight structural definition).
- **lddt**: Pairwise LDDT score, averaged over all pairs. We consider it a sequence identity score.

It is hypothesised that protein sequences can have very low sequence identity, but still exhibit similar 3D structures and biological functions. (Remote Homology Detection/Discovery).

- **pfam_score**: Value from 0 to 1 indicating how "pure" the Metacluster is with respect to Pfam-36 labels.
- **pfam_labels**: List of Pfam clans/families found within this Metacluster (A maximum of 5 clans or families is reported).

Table 2: **DPCStruct Representative Sequences** : We parse sequences from file 2, which are representatives(maximum 2) for each metacluster, detailed in file 1.

	mc_id	uniprot_protein_id	seq_range	seq_length	aa_seq
0	MC0	A0A537IMV5	3-112	110	KIFICFALCLLAFFKPASAQFSTNENIKDQPKWGLAGQKYVEYYY...
1	MC0	A0A1Q3ZAL7	4-118	115	LIFLFVLTTSFTTNLFKAQVNVNININSQPEWGPGRGYNYVESYYL...
2	MC1	A0A7Y5H9I3	1-65	65	MKEVVSVAVLASKEGFNVYDLADGHTIKMKTIVLDVVRVEGVKDEL...
3	MC1	A0A497KZ75	9-72	64	SVDLDFTEVEEHWCYKLSDGTTLVKVLVLRGVKRLNRYEPDGTPI...
4	MC10	A0A182KBP7	1-56	56	TLIPSAPQNFTVEKILPSTDGTFLALAGPKGVSIHELPRRWGPGSQ...

Where:

- **mc_id** : Metacluster ID.
- **uniprot_protein_id** : Representative (0, 1, or 2) Protein Uniprot IDs.
- **seq_range** : Domain start - Domain end, indicating where in the protein sequence this structural domain is located.
- **seq_length** : Amino-acid sequence length.
- **prot_seq** : Sequence of amino acids of each representative.
- **structure_3d**: We will add this column using the PDB file per representative (if any) to display the 3D structure, using file 3.

From file 5 to file 9, the general naming is

annotated_[cath|scop]_qc[x]_t[x]_l[x].tsv ; related to the comparison between DPC-struct and CATH or SCOP:

- **qc**: Query coverage threshold (**0.5** or **0.8**). This parameter indicates how much of the CATH/SCOP domain (the query) is covered by the DPCStruct representative (the target). The authors considered DPCStruct representatives where the query coverage is at least 0.5.
- **t**: Template modelling score threshold (**0.5**). This parameter is a standard measure to assess the global topological similarity of the protein folds. We aim to determine whether two proteins share a fold. In the paper, the authors used a threshold of $t = 0.5$, meaning they only care about matches where the overall 3D shape is almost the same.
- **l**: LDDT (Local Distance Difference Test) threshold (**0.5**). This parameter is a score assessing whether local atomic interactions are preserved, thus validating local structural quality. While TM-Score handles the "big picture" (the backbone path), LDDT addresses the "details" (are the side chains and local atoms in roughly the same relative positions?). A high TM-score with a low LDDT indicate that the shape is similar, but the actual chemistry or local packing is distorted. Therefore, the authors set $l = 0.5$ to ensure the match is satisfactory at the local level as well.

Thus, each of the four files states: **"Show me matches that are definitely the same fold ($TM > 0.5$), look locally correct ($LDDT > 0.5$), and cover at least half of the known structure ($QC > 0.5$)."**

1. CATH

For the comparison with CATH, where the authors used 0.5 and 0.8 as the query coverage thresholds, it turned out that the subset for the latter threshold is included in the former (Figure 1):

```

1 # We check whether annotated_cath_qc0.8_t0.5_l0.5.csv is a subset of annotated_cath_qc0.5_t0.5_l0.5.csv
2 set_cath5 = set(df_cath5["query"])
3 set_cath8 = set(df_cath8["query"])
4 print("Is annotated_cath_qc0.8_t0.5_l0.5.csv a subset(sub dataframe) of annotated_cath_qc0.5_t0.5_l0.5?", set_cath8.issubset(set_cath5))
5
6 # We check the intersection size
7 print(f"annotated_cath_qc0.5_t0.5_l0.5 size: {len(set_cath5)}")
8 print(f"annotated_cath_qc0.8_t0.5_l0.5 size: {len(set_cath8)}")
9 print(f"Intersection: {len(set_cath8.intersection(set_cath5))}")

```

[6] ✓ 0.0s Python

```

... Is annotated_cath_qc0.8_t0.5_l0.5.csv a subset(sub dataframe) of annotated_cath_qc0.5_t0.5_l0.5? True
annotated_cath_qc0.5_t0.5_l0.5 size: 1311
annotated_cath_qc0.8_t0.5_l0.5 size: 1274
Intersection: 1274

```

Figure 1: File 6 is included in file 5.

After an agreement on whether we should consider the stricter subset (small, but with near "full" domain matches) or the larger one (less strict, partial matches), the interesting properties could be summarised as shown in Figure 2:

	query	target	qstart	qend	tstart	tend	qlen	tlen	qcov	tcov	alnlen	pident	qtmcscore	ttmscore	alntmscore	lddt
0	4.10.270	A0A7F8Q518_39405	1	55	127	181	68	182	0.81	0.30	55	61.8	0.58	0.25	0.25	8.906E-01
1	2.170.210	A0A7J6I9T2_8385	25	168	35	172	172	180	0.84	0.77	156	12.1	0.22	0.21	0.21	5.044E-01
2	3.40.20	A0A0D8XCM8_6520	9	134	15	153	134	171	0.94	0.81	155	12.2	0.23	0.20	0.20	-NAN
3	3.90.105	A0A126G1X7_6894	2	58	96	160	58	281	0.98	0.23	69	18.8	0.83	0.18	0.18	8.005E-01
4	1.20.1740	A0A2J6WLT6_15600	11	417	2	411	435	412	0.94	1.00	427	21.7	0.82	0.87	0.87	6.804E-01

Figure 2: CATH vs DPCStruct : Meaningful properties.

Where :

- **query**: Identifier of the query(the one we are searching with) protein structure,from the CATH database.
- **target**: Identifier of the target(the match found in our database) protein structure in DPCstruct named as **proteinID_MCID**
- **qstart**: Starting residue position of the alignment on the query sequence
- **quend**: Ending residue position of the alignment on the query sequence
- **tstart**: Starting residue position of the alignment on the target sequence
- **tend**: Ending residue position of the alignment on the target sequence

- **qlen**: Total length(number of amino acids) of the query protein
- **tlen**: The total length of the target protein.
- **qcov**: Query coverage. The fraction(0 - 1) of the query sequence that is covered by the alignment
- **tcov**: Target coverage. The fraction(0 - 1) of the target sequence that is covered by the alignment
- **alnlen**: Alignment length. The total number of aligned residue pairs.
- **pident**: Percentage identity. The percentage of identical amino acids in the aligned region.
- **qtmscore**: TM-score(Template Modeling score) normalized by the query length. It measures the structural similarity between two structures.
- **ttmscore**: TM-score normalized by the target length.
- **alntmscore**: TM-score normalized by the alignment length.
- **l1ddt**: Local Distance Difference Test score. A superposition-free score that evaluates local structure preservation (0 to 1). Higher is better

Our list is still quite long, we may shorten it by grouping some parameters:

1. **Structural Similarity** (The "Quality" of the Match)

- **qtmscore** (Query TM-score): This is arguably the most important metric. It tells us exactly how well the CATH fold (query) is reproduced by the DPCStruct Metacluster. The higher the score, the better the match.
- **l1ddt**: Measures local structural agreement. It confirms that the backbone isn't just roughly the same shape, but that the local atomic packing is also correct.

2. **Coverage** (The "Completeness" of the Match)

- **qcov** (Query Coverage): The percentage of the CATH domain covered by the alignment. If qcov is high (>0.8), DPCStruct isn't just finding fragments—it's finding complete domains.
- **tcov** (Target Coverage): The percentage of the DPCStruct representative covered by the alignment. If tcov is low but qcov is high, it suggests our DPCStruct representative is a multi-domain protein or a larger structure that contains the CATH fold within it.

3. **Sequence vs. Structure** (The "Discovery" Value)

- **pident** (Percent Identity): The sequence similarity between the two. This is where the magic happens (Figure 3):

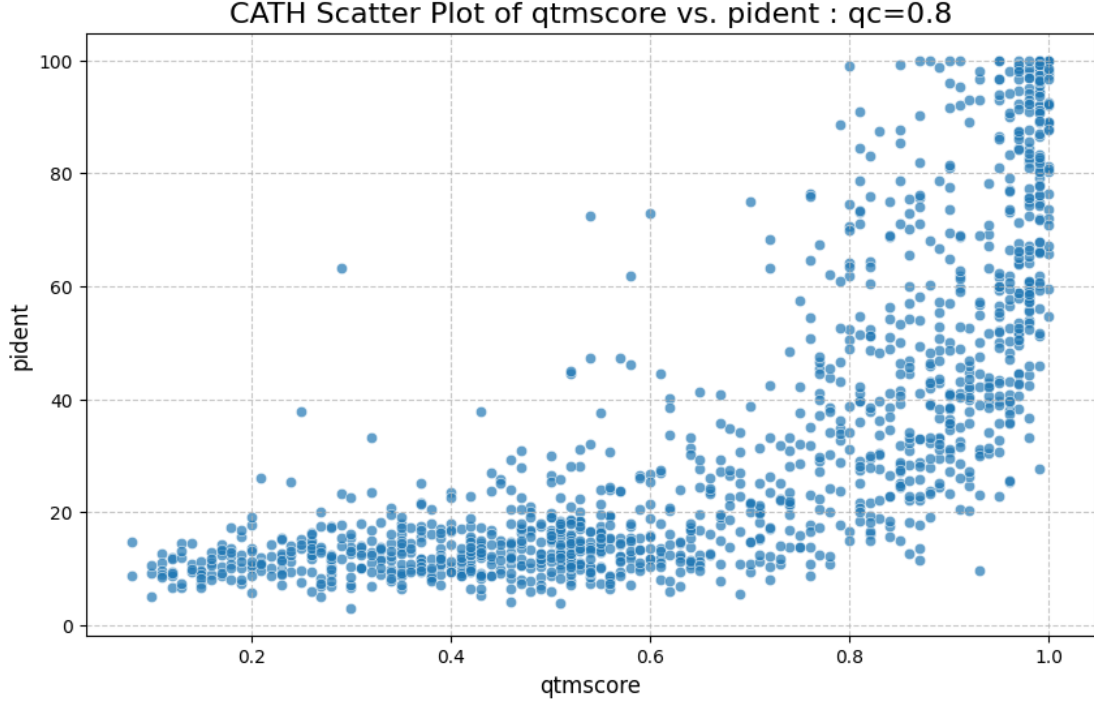


Figure 3: Strong positive correlation (0.75) between qtmscore (structural similarity) and pident (sequence identity).

From Figure 3, we notice that as the percentage identity (pident) increases, the structural similarity score (qtmscore) tends to increase significantly. This aligns with the biological expectation that sequences with higher identity generally share more similar structures. Therefore, statistically, sequence identity is a strong predictor of structural similarity. However, there is still significant structural conservation even when sequence identity drops; having the same structure doesn't guarantee the similarity at a sequence level.

2. SCOP

For SCOP, I also expected file 8(qc=0.8) to be included in file 7(qc=0.5). Unfortunately(Figures 4 and 5):

```

1 # to check whether annotated_scop_qc0.8_10.5_10.5.csv is a subset of annotated_scop_qc0.5_10.5_10.5.csv
2 set_scop5 = set(df_scop5["query"])
3 set_scop8 = set(df_scop8["query"])
4 print("Is annotated_scop_qc0.8_10.5_10.5.csv a subset(sub dataframe) of annotated_scop_qc0.5_10.5_10.5?", set_scop8.issubset(set_scop5))
5
6 # to check the intersection
7 print("annotated_scop_qc0.5_10.5_10.5 size: (len(set_scop5))")
8 print("annotated_scop_qc0.8_10.5_10.5 size: (len(set_scop8))")
9 print("Intersection: (len(set_scop8.intersection(set_scop5)))")

```

(4) ✓ 00s

```

--- Is annotated_scop_qc0.8_10.5_10.5.csv a subset(sub dataframe) of annotated_scop_qc0.5_10.5_10.5? False
annotated_scop_qc0.5_10.5_10.5 size: 1411
annotated_scop_qc0.8_10.5_10.5 size: 1399
Intersection: 1258

```

Figure 4: File 8 $\subsetneq$ File 7

```

1 # Investigate queries present in SCOP 0.8 but not in SCOP 0.5
2 missing_queries = set_scop8 - set_scop5
3 print(f"Number of queries in SCOP 0.8 but not in SCOP 0.5: {len(missing_queries)}")
4
5 # Look at the data for these missing queries in SCOP 0.8
6 missing_in_scop8_df = df_scop8[df_scop8["query"].isin(missing_queries)]
7 print(f"\nSample of queries in SCOP 0.8 but not SCOP 0.5:")
8 display(missing_in_scop8_df.head())
9
10 # Check the distribution of qcov for these missing queries
11 print("\nqcov stats for these missing queries:")
12 print(missing_in_scop8_df["qcov"].describe())
13
14 # Check the qcov distribution for the whole datasets to confirm our assumption about "qc"
15 print("\nSCOP 0.5 qcov min:", df_scop5["qcov"].min())
16 print("\nSCOP 0.8 qcov min:", df_scop8["qcov"].min())

```

Number of queries in SCOP 0.8 but not in SCOP 0.5: 181

Sample of queries in SCOP 0.8 but not SCOP 0.5:

	query	target	qstart	qend	tstart	tend	qlen	tlen	qcov	tcov	alnlen	pidcnt	evalue	bits	qtm score	ttmscore	alntmscore	lddt	prob
5	P25080_2000005	A0ATC1T382_35185	1	179	5	183	179	183	1.000	0.978	179	65.3	4.271000e-29	1164	0.9912	0.9697	0.9697	0.9782	1.0
7	P29372_2000011	A0AF9D7R9_7641	1	157	13	169	157	213	1.000	0.737	157	88.5	1.813000e-29	1146	0.9788	0.7241	0.7241	0.9680	1.0
24	P04062_2000031	S0XU2_34978	1	354	15	368	354	379	1.000	0.934	354	92.3	8.702000e-72	2929	0.9995	0.9336	0.9336	0.9987	1.0
29	Q9WY59_2000038	A0A38QA21_9733	1	280	4	283	281	490	0.996	0.571	280	58.4	1.217000e-36	1628	0.9878	0.5882	0.5882	0.9530	1.0
34	P54512_2000044	A0A7X4FJ3_11647	1	76	134	211	77	216	0.987	0.361	78	17.9	9.371000e-02	100	0.7962	0.3182	0.3182	0.7981	1.0

Qcov stats for these missing queries:

```

count    181.000000
mean      0.538981
std       0.646980
min       0.800000
25%      0.887000
50%      0.967000
75%      0.995000
max       1.000000
Name: qcov, dtype: float64

```

SCOP 0.5 qcov min: 0.5
SCOP 0.8 qcov min: 0.8

Figure 5: We may need to investigate to understand the reason behind this.

Focusing on strict matches($qc=0.8$), we summarise relevant features in Figure 6:

	query	target	qstart	qend	tstart	tend	qlen	tlen	qcov	tcov	alnlen	pidcnt	qtm score	ttmscore	alntmscore	lddt	prob
0	R4YEY9_2000000	A0A6N8Q492_4003	1	49	208	256	49	270	1.00	0.18	49	97.9	1.00	0.18	0.18	1.00	
1	P39805_2000001	R6QN42_48898	1	54	1	54	55	68	0.98	0.79	54	50.0	0.96	0.78	0.78	0.95	
2	U5KQB1_2000002	A0A1K0FU75_8415	1	134	29	162	134	163	1.00	0.82	134	87.3	1.00	0.82	0.82	1.00	
3	Q9X1H2_2000003	A0A7C2KEC5_36025	1	186	1	186	203	186	0.92	1.00	186	97.8	0.92	1.00	1.00	0.95	
4	Q87WW1_2000004	A0A0P9IV70_61195	1	105	105	209	105	213	1.00	0.49	105	99.0	1.00	0.49	0.49	1.00	

Figure 6: SCOP($qc=0.8$) relevant features.

How does structural similarity varies with respect to sequence similarity?:

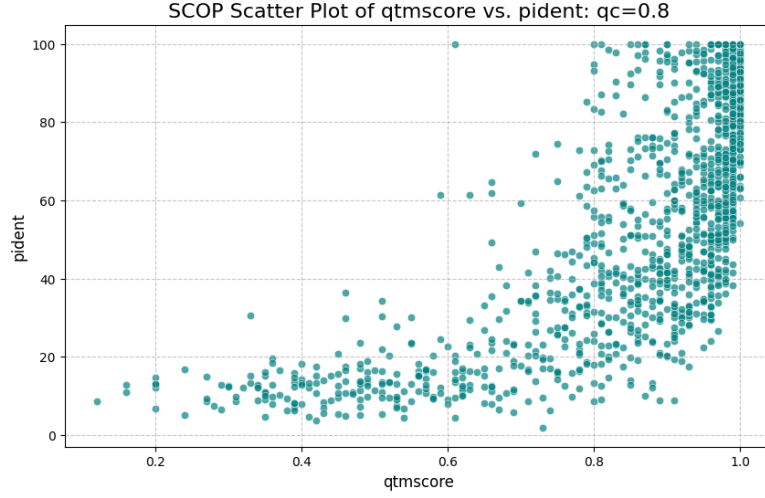


Figure 7: Strong positive correlation(0.73) between qtmscore(3D folds) and pident(sequences). However, structural similar doesn't imply similarity at the sequence level.

3 Conclusion

Putting everything together, here is how to integrate DPCstruct within our web app:

1. When a user has chosen DPCstruct from the landing page and entered a valid ID, we display five tables (Table 1 : Metaclusters properties (Tab 1), Table 2 : Representative sequences (Tab 2), Table 3 : 3D viewer (we display a 3D structure (using File 3), Table 4: Comparison with CATH and SCOP (we filter matching rows in Figure 2 and Figure 6), Table 5: Downloads(MCID.pdb and MCID.fasta files)
2. When a user has chosen PFAM and a valid ID (family or clan), we display matching samples (File 10). However, in contrast to DPCFam, we have both families and clans, and instead of the average overlap, we have scores.
3. When a user has chosen Uniprot + valid protein ID, for DPCstruct, we have ranges for representatives only.
4. For the Metaclusters/DPCstruct URL, we can modestly display Metaclusters properties (Table 1)
5. For the Downloads/DPCstruct URL, we may merely provide access to all pdbs, all fasta, and some other relevant CSV files.

Do we need to extend the selection button on the landing page so that users can also choose CATH/SCOP, enter a fold ID, and then see matching metaclusters?

References

Barone, F., Laio, A., Punta, M., Cozzini, S., Ansuini, A., and Cazzaniga, A. (2025). Unsupervised domain classification of alphafold2-predicted protein structures. *PRX Life*, 3(2):023009.